

Analysis of consistency violating faults in distributed graph algorithms

Kamal Giri, Amit Garu

Abstract—With the increased utilization of distributed systems, there has been a greater focus on making these systems more resilient to faults, particularly consistency violation faults (*cvfs*) which occur when neighboring nodes read incorrect or stale data and base their actions on this erroneous information. While previous studies [3] have shown that tolerating *cvfs* has minimal impact on program performance, analyzing the overhead of *cvfs* becomes crucial when dealing with large program states. In this paper, we compute the program transitions of two different self-stabilizing algorithms and use the rank effect to assess the impact of *cvfs*. Additionally, we quantitatively evaluate the impact of *cvfs* on individual nodes within these algorithms to understand the varying costs associated with *cvfs* across different nodes in a distributed system.

Index Terms—Distributed system, consistency violation faults, program transition, rank effect

I. INTRODUCTION

Distributed systems consist of multiple processors dispersed across a network, working in concert to achieve a common objective. However, this coordination comes with the challenge of synchronization. Insufficient synchronization can lead to consistency violation faults (*cvfs*), such as simultaneous data access by multiple nodes [3]. While methods such as local mutual exclusion and atomic operations can mitigate these *cvfs*, their implementation incurs significant costs, which can adversely affect program performance. As an alternative, *cvfs* tolerance strategies allow programs to tolerate *cvfs* and converge to a stable state after a finite number of steps in self-stabilizing systems.

Although tolerating *cvfs* can be advantageous, it is crucial to evaluate the associated overhead, especially in systems with large state spaces. Experimental analysis using real-life examples can be time-consuming and impractical due to the complexity and size of the state space, making it difficult to replicate and measure all possible *cvfs* and their associated costs. Alternatively, analyzing program transitions offers a systematic approach to evaluate the impact of *cvfs* by assessing all potential transitions within a given system. This method allows for a quantitative analysis of *cvfs* costs based on the state space of the program. However, this approach also presents challenges, as increasing the number of nodes in the system can result in a vast number of program transitions, rendering calculations and storage of all possible transitions infeasible.

II. METHODOLOGY

A. Distributed Program

A distributed, parallel, or concurrent program consists of a set of n processes [3]. Each process j has a set of neighboring processes, denoted as N_{b_j} .

Each process is associated with a set of variables and a set of actions. The variables of the program p are the union of all variables across the processes, and the state of the program p is defined by assigning values to all the variables of the processes.

The state space S_p of the program comprises all possible states of p . Transitions occur between states, known as program transitions. A transition in a program is denoted by

$$\{(s_0, s_1) \mid s_0, s_1 \in S_p\},$$

where s_1 is obtained by executing an action on s_0 [3].

B. Self-stabilization

The program is called self-stabilizing if it converges to the final legitimate state from any arbitrary state without any outside intervention.

Let p be the program and inv be the state predicate of p . We say that p is self-stabilizing if and only if:

- 1) For any program transitions $\langle s_0, s_1, \dots \rangle$, there exists a state s_i such that $s_i \in \text{inv}$.
- 2) If (s_0, s_1) is a transition of p and $s_0 \in \text{inv}$, then $s_1 \in \text{inv}$ [3].

These two properties of a self-stabilizing algorithm are known as the convergence property and the closure property. In other words, if a program is self-stabilizing, it will converge to a legitimate state from any arbitrary initial state, and once it reaches a legitimate state, it will remain in that state indefinitely.

C. Consistency violating faults

Consistency violation faults occur when the program p reads an incorrect value of a variable assigned to one of its states. Let $\langle s_0, s_1, s_3, \dots \rangle$ represent the program computation, and (s_0, s_1) denote the program transition.

Let the action of the process be $g \rightarrow s_t$. For the program transition (s_0, s_1) to take place, g needs to be evaluated to true in the state s_0 , and s_1 is obtained by executing s_t . However, the execution of $g \rightarrow s_t$ involves the variables of process j . During this action, it is possible that, without proper synchronization, g or s_t may use outdated variables [3]. This leads to consistency violation faults.

In this paper, we simulate such behaviors and calculate the cost of such *cvfs*.

D. Programs of consideration

In this paper, I have used two self-stabilizing algorithms: Dijkstra's Token Ring Problem [1] and Graph Coloring [2].

Dijkstra's Token Ring Problem: In this algorithm, nodes $0, 1, \dots, n-1$ are arranged in a ring, so each node has both a left and a right neighbor. Each process can hold a value from the set $\{0, 1, 2\}$. The program is self-stabilizing if and only if exactly one node is designated as eligible according to the algorithm's criteria. If no node is eligible, the program updates the value of each process based on the values of its neighboring nodes.

For node 0: if $(val(0)+1) \bmod 3 = val(1)$, then $val(0) = (val(0) - 1) \bmod 3$.

For node $n-1$: if $val(n-2) = val(0)$ and $(val(n-2)+1) \bmod 3 \neq val(n-1)$, then $val(n-1) = (val(n-2) + 1) \bmod 3$.

For a node $j, 0 < j < n-1$: if $(val(j) + 1) \bmod 3 = val(j-1)$, then $val(j) = val(j-1)$. Additionally, if $(val(j) + 1) \bmod 3 = val(j+1)$, then $val(j) = val(j+1)$.

Arbitrary Graph Coloring: In this algorithm, any arbitrary graph is selected for vertex coloring, where each process j of the graph is associated with a color c_j based on the colors of its neighbors. Process j agrees with the algorithm if the color c_j of the process is different from the colors c_i in N_{c_j} , where N_{c_j} is the set of colors of the neighbors of process j , and $c_i = \min(\{0, \dots, B\} - N_{c_j})$, where $\{0, \dots, B\}$ is the set of all possible colors [3]. Hence, $c_j \in \{0, \dots, B\}$, if $c_j \neq \min(\{0, \dots, B\} - N_{c_j})$, then $c_j = \min(\{0, \dots, B\} - N_{c_j})$.

In this paper, we first obtain the full program configuration for the given self-stabilizing algorithm. These states are experimentally generated rather than based on actual state transitions during program runtime. Next, we generate all the invariants and illegitimate states and rank all the program states. The ranking is based on the number of steps each state takes to reach an invariant. Consequently, all states that are invariants are assigned rank 0. For illegitimate states, we compute both the average and maximum rank, where the maximum rank represents the longest path required to reach an invariant.

The maximum rank, $\max rank(s_0)$, is defined as:

$$\max rank(s_0) = \max rank(s_1) + 1 \text{ for } (s_0, s_1)$$

In other words, the maximum rank of s_0 is calculated based on the ranks of its successors which are already known.

We also calculate the average rank using:

$$\text{average rank} = \frac{\text{total path length}}{\text{path count}}$$

where,

path count = number of paths from a state s to reach the invariant, and,

total path length = sum of the lengths of all paths [3].

Following this, we compute the program transition rank, which ranks all states in the program state space. The program transition rank effect measures the cost added to the program when a state transition is either a program transition or a *cvf*. The rank effect can be either negative or positive, indicating the nature of the state transition. Program transition rank

effects are always negative because these transitions converge the program to an invariant. In contrast, *cvfs* result in a positive rank effect, as they are incoming *cvfs* calculated by perturbing the invariant. The rank effect is given by:

$$\text{rank effect}(\text{dest}) = \text{rank}(\text{dest}) - \text{rank}(\text{start})$$

Additionally, we calculate *cvfs* out rank effects, which can be both positive and negative. These are computed by perturbing the illegitimate state, which can result in either convergence (negative rank effect) or greater divergence (positive rank effect).

Finally, we assess the impact of *cvfs* on individual nodes. Previously, we calculated the rank effect by analyzing all program states, program transitions, and perturbations. This approach provided an overall impact of *cvfs* on the entire system but did not address how *cvfs* affect individual nodes across different self-stabilizing algorithms. To address this, we tracked the perturbation positions during each *cvfs* event. This data will later be used to evaluate the specific impact of *cvfs* on individual nodes, offering insights into how different self-stabilizing algorithms are affected by *cvfs* at the node level.

III. RESULTS AND DISCUSSION

The project code is available on GitHub: https://github.com/kgiri2000/cvf_node_effect_analysis.git.

A. Graphs

A circular graph with 10 nodes for Dijkstra's Token Ring Problem and a 7-node graph for Graph Coloring are used in this case study, as shown below:

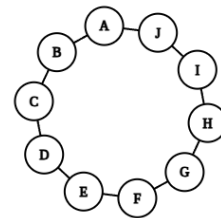


Fig. 1. Circular graph with 10 nodes used in Dijkstra's Token Ring problem

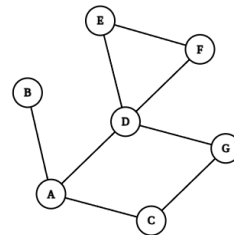


Fig. 2. Arbitrary graph with 7 nodes used for Graph Coloring problem

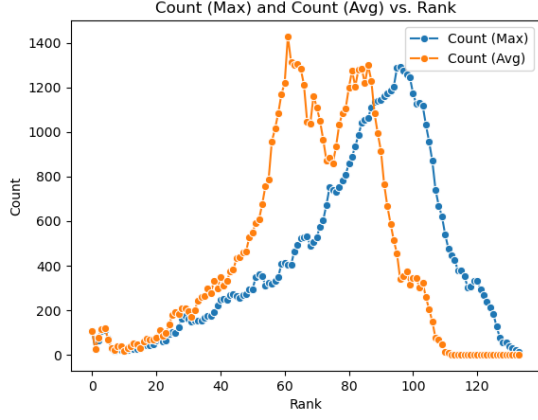


Fig. 3. Rank vs Program states count for Token Ring Problem with 10 nodes

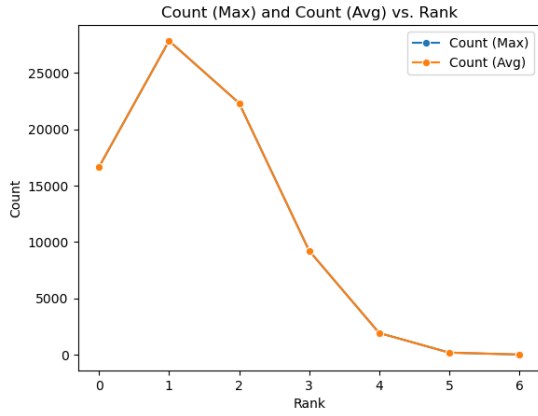


Fig. 4. Rank vs Program states count for Graph Coloring with 7 nodes

B. Rank Count

In Figures 3 and 4, we plot the count of states with similar ranks relative to their rank values.

Figure 3 demonstrates that the rank ranges from 0 to 133, with a high number of states falling within the rank range of 60 to 100. This indicates a broad distribution of ranks with a concentration of states around the middle range.

In contrast, Figure 4 shows that for graph coloring, the rank ranges from 0 to 6, with most states having ranks between 0 and 3. This narrower range of ranks suggests that the graph coloring algorithm converges more quickly compared to the other algorithm. Due to state space explosion, we could not generate graphs for more than 7 nodes in the graph coloring scenario.

These figures illustrate that graph coloring is likely to converge more quickly, given its lower rank values and the concentrated distribution of states with similar ranks.

C. Rank Effect for Program Transitions and *cvfs*

Examining the graphs for both the token ring and graph coloring algorithms reveals similar distributions. However, as previously discussed, the rank effect range for Dijkstra's token ring algorithm is significantly higher. This high rank effect indicates that a greater program transition rank leads

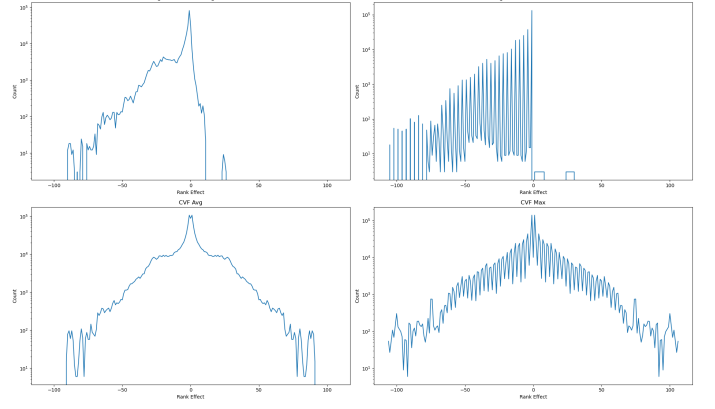


Fig. 5. Program Transition and *cvfs* rank effect plot for Dijkstra's token ring problem with 10 nodes

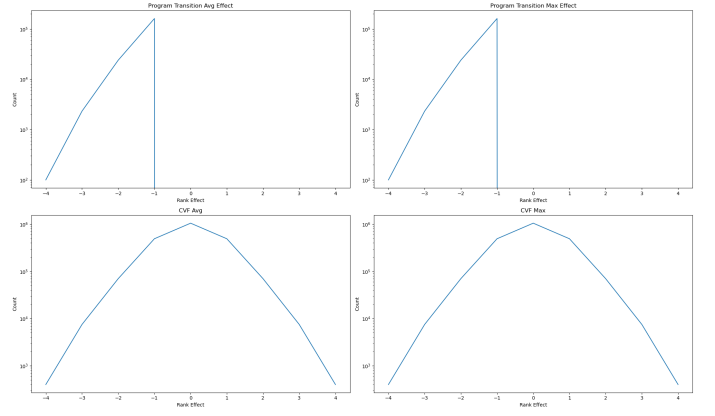


Fig. 6. Program Transition and *cvfs* rank effect plot for Graph Coloring problem with 7 nodes.

to longer convergence times, and similarly, a high *cvfs* rank effect results in extended convergence times when *cvfs* occur. Additionally, it is noteworthy that the rank effects exhibit substantial fluctuations at the endpoints of the Dijkstra's token ring problem.

D. Impact of *cvfs* on individual nodes

Examining Figure 2 for graph coloring, we observe that Node D, with a degree of 4, has a significant impact on *cvfs* counts as seen in Figure 8. In contrast, Node B, with a degree of 1, shows a relatively low rank effect, and Node A, with a degree of 3, follows a similar pattern. This suggests a positive correlation between node degree and the *cvfs* effect it can have.

In Figures 1 and 7, which illustrate the Dijkstra's Token Ring problem, the data show that the nodes at the bottom exhibit the lowest *cvfs* effects, while nodes just above them exhibit the highest *cvf* effects. This indicates that *cvfs* occurring in nodes closer to the bottom have a greater impact on convergence compared to those affecting the top nodes.

IV. CONCLUSION

Although our analysis focused on each graph for both the graph coloring and Dijkstra's token ring problems, we

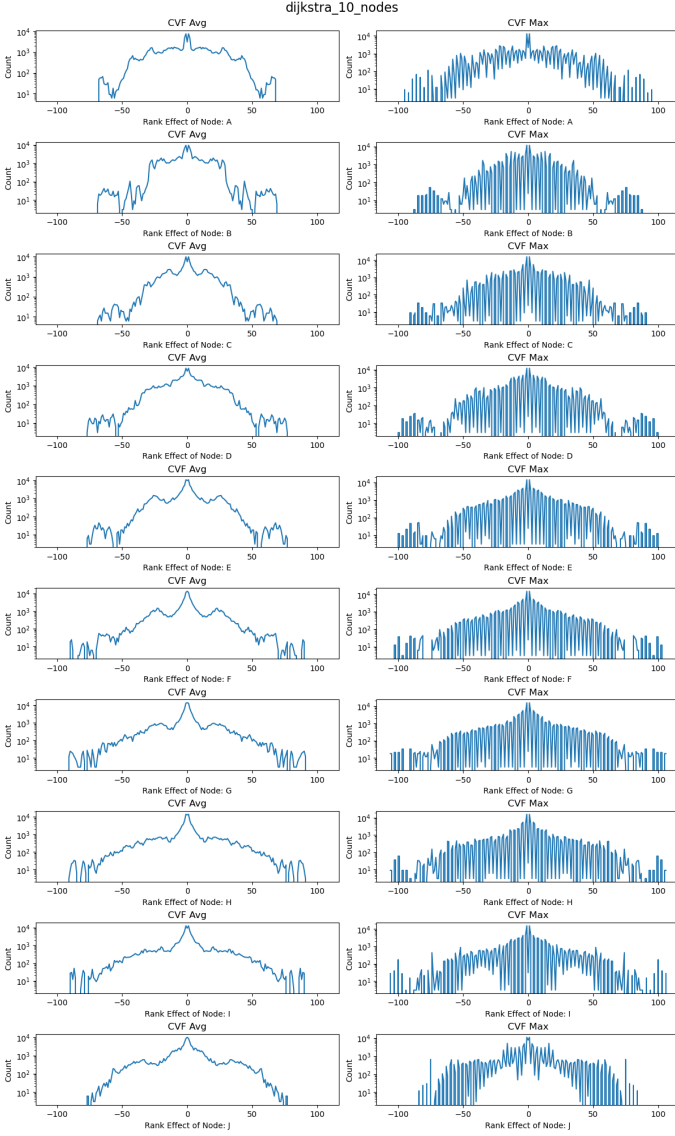


Fig. 7. Effect of *cvfs* on individual nodes using Dijkstra's token ring problem with 10 nodes

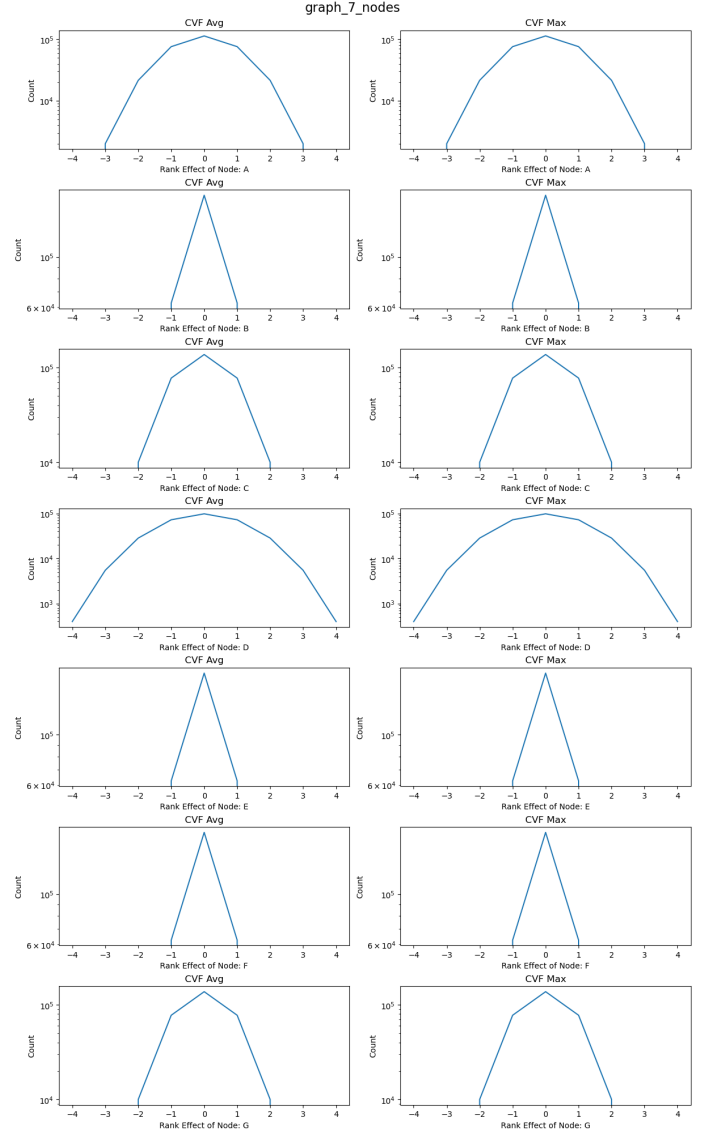


Fig. 8. Effect of *cvfs* on individual nodes using Graph Coloring with 7 nodes

identified some notable correlations. For graph coloring, we observed a correlation between the node degree and the accumulated *cvf* effect, while for the token ring problem, we found a correlation between *cvf* effects and node positions.

Comparing the two algorithms, we first calculated the rank count for all program states. The results indicated that states in the token ring problem generally have higher rank values compared to those in graph coloring. Both algorithms produced similar patterns in program transitions and *cvf* rank effects, suggesting they converge and diverge in comparable ways. However, due to its higher rank values, the token ring problem typically experiences longer convergence times. Finally, we discussed the impact of *cvfs* on individual nodes and found a correlation between node degree and *cvf* effects for graph coloring, and a correlation between node position and *cvf* effects for the token ring problem

REFERENCES

- [1] Edsger W. Dijkstra. Self-stabilizing systems in spite of distributed control. *Commun. ACM*, 17(11):643–644, 1974.
- [2] Maria Gradinariu and Sébastien Tixeuil. Self-stabilizing vertex coloration and arbitrary graphs. In *Proceedings of the 4th International Conference on Principles of Distributed Systems, OPODIS 2000, Paris, France, December 20-22, 2000*, pages 55–70, 2000.
- [3] Duong Ngoc Nguyen, Arya Tanmay Gupta, and Sandeep Kulkarni. Analyzing program transitions to compute benefit of tolerating consistency violation faults. In *Proceedings of the 24th International Conference on Distributed Computing and Networking*, pages 58–69, 2023.